

Foundations of Language Modeling

Chapter 7: Scaling Up to GPT and the ChatGPT Pipeline

1 Breaking Permutation Invariance

If we analyze the Self-Attention and Feed-Forward networks we have built thus far, a critical issue emerges: they possess no inherent concept of space or sequence order.

Remark 1.1 (Permutation Invariance of Attention). *In a scaled dot-product attention block, if we randomly shuffle the order of the input vectors in the sequence, the attention mechanism will simply aggregate the exact same vectors in the exact same proportions (subject to the causal mask). The neural network treats the input as a "bag of words" rather than an ordered sequence.*

To fix this, we must inject information about the position of each token. We do this by utilizing **Positional Embeddings**.

Definition 1.2 (Positional Embeddings). *Let T be the block size and C be the embedding dimension. We define a positional embedding matrix $\mathbf{P} \in \mathbb{R}^{T \times C}$. For a token at position $t \in \{1, \dots, T\}$, its positional embedding is the row vector $\mathbf{p}_t = \mathbf{P}_{t,*}$. The final input representation $\mathbf{x}_t^{\text{in}}$ is the element-wise sum of the token embedding and the positional embedding:*

$$\mathbf{x}_t^{\text{in}} = \mathbf{W}_{\text{token}}(x_t) + \mathbf{p}_t \quad (1)$$

Intuition: By adding these vectors, the embedding space holds dual information. The neural network learns to disentangle "what" the token is (from the token embedding) and "where" the token is (from the positional embedding).

2 Assembling the Full GPT Architecture

With all pieces in place, we can construct the full Generative Pre-trained Transformer (GPT). The architecture consists of:

1. A Token Embedding Table.
2. A Positional Embedding Table.
3. A sequence of N stacked Transformer **Blocks** (Self-Attention + Feed-Forward).
4. A final Layer Normalization step (`ln_f`).
5. A linear projection head (`lm_head`) mapping the C -dimensional output back to the vocabulary size V to compute the logits.

3 From Pre-Training to ChatGPT

The model we have programmed is a "base model." It is trained purely to predict the next token on a massive corpus of text (Pre-training). If you prompt a base model with a question, it might answer it, or it might just generate a structurally similar question, because it is merely a *document completer*.

To transform a base model into an assistant like ChatGPT, OpenAI utilizes a three-step fine-tuning pipeline, specifically **Reinforcement Learning from Human Feedback (RLHF)**:

1. **Supervised Fine-Tuning (SFT)**: Human contractors write thousands of prompt-response pairs. The base model is fine-tuned on this dataset using the exact same next-token prediction logic. The model learns to adopt the structural format of an "assistant."
2. **Reward Modeling (RM)**: The SFT model generates multiple different responses to a single prompt. Humans rank these responses from best to worst. A new neural network (the Reward Model) is trained to predict the human preference score for any given text.
3. **Proximal Policy Optimization (PPO)**: The SFT model generates responses, which are scored by the Reward Model. Using PPO (a Reinforcement Learning algorithm), the language model's weights are updated to maximize the expected reward, deeply aligning its outputs with human preferences.

4 Worked Examples and Solved Exercises

4.1 Mathematical Exercise: Parameter Counting

Problem: Consider a miniaturized GPT model with the following hyperparameters:

- Vocabulary size $V = 65$
- Block size (context length) $T = 8$
- Embedding dimension $C = 32$
- Number of attention heads $h = 4$
- Number of Transformer blocks $N_L = 3$

Assuming biases are disabled in all linear layers *except* the Feed-Forward network, calculate the exact number of learnable parameters in this model (ignore the parameters inside the LayerNorms for simplicity).

Solution: We compute the parameter count layer by layer.

1. Embeddings:

- Token Embedding ($\mathbf{W}_{\text{token}} \in \mathbb{R}^{V \times C}$): $65 \times 32 = 2,080$
- Positional Embedding ($\mathbf{P} \in \mathbb{R}^{T \times C}$): $8 \times 32 = 256$

2. A Single Transformer Block:

- *Self-Attention:* Contains Query, Key, and Value projections ($\mathbb{R}^{C \times C}$ each), plus an Output projection ($\mathbb{R}^{C \times C}$).

$$4 \times (32 \times 32) = 4 \times 1024 = 4,096 \text{ parameters.}$$

- *Feed-Forward:* Expands C to $4C$, then projects $4C$ back to C . Includes biases.

$$\text{Layer 1 } (\mathbf{W}_1, \mathbf{b}_1) : (32 \times 128) + 128 = 4,096 + 128 = 4,224$$

$$\text{Layer 2 } (\mathbf{W}_2, \mathbf{b}_2) : (128 \times 32) + 32 = 4,096 + 32 = 4,128$$

$$\text{Total for FFN} = 4,224 + 4,128 = 8,352 \text{ parameters.}$$

- Total per block = $4,096 + 8,352 = 12,448$ parameters.

Since there are $N_L = 3$ blocks, the total block parameters = $3 \times 12,448 = 37,344$.

3. Final Linear Head: Maps embedding dimension back to vocabulary size ($\mathbb{R}^{C \times V}$), typically without bias in GPT.

$$32 \times 65 = 2,080 \text{ parameters.}$$

Total Parameters:

$$P_{\text{total}} = 2080(\text{TokEmb}) + 256(\text{PosEmb}) + 37344(\text{Blocks}) + 2080(\text{Head}) = \mathbf{41,760}$$

Note: GPT-3, for comparison, has 175,000,000,000 parameters!

4.2 Coding Example: The Full GPTLanguageModel Class

Problem: Implement the final ‘GPTLanguageModel’ in PyTorch, tying together the embedding tables, a sequence of ‘Block’ modules, the final ‘LayerNorm’, and the ‘lm_head’.

Solution:

```
1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4
5  class GPTLanguageModel(nn.Module):
6  def __init__(self, vocab_size, n_embd, n_head, n_layer, block_size, dropout):
7  super().__init__()
8  self.block_size = block_size
9
10 # 1. Embedding tables
11 self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
12 self.position_embedding_table = nn.Embedding(block_size, n_embd)
13
14 # 2. Sequence of Transformer Blocks
15 # nn.Sequential allows us to stack multiple blocks cleanly
16 self.blocks = nn.Sequential(*[
17 Block(n_embd, n_head, block_size, dropout) for _ in range(n_layer)
18 ])
19
20 # 3. Final LayerNorm and Output Head
21 self.ln_f = nn.LayerNorm(n_embd) # Final norm before decoding
22 self.lm_head = nn.Linear(n_embd, vocab_size)
23
24 def forward(self, idx, targets=None):
25 B, T = idx.shape
26
27 # Retrieve token embeddings and positional embeddings
28 tok_emb = self.token_embedding_table(idx) # (B, T, C)
29 pos_emb = self.position_embedding_table(torch.arange(T, device=idx.device)) #
30 (T, C)
31
32 # Element-wise addition; PyTorch broadcasts pos_emb to (B, T, C)
33 x = tok_emb + pos_emb
34
35 # Pass through the transformer blocks
36 x = self.blocks(x) # (B, T, C)
37
38 # Apply final layer norm
39 x = self.ln_f(x) # (B, T, C)
40
41 # Project back to vocabulary dimension to get logits
42 logits = self.lm_head(x) # (B, T, vocab_size)
43
44 if targets is None:
45 loss = None
46 else:
47 B, T, C = logits.shape
48 logits = logits.view(B*T, C)
49 targets = targets.view(B*T)
50 loss = F.cross_entropy(logits, targets)
51
52 return logits, loss
```